Table des matières

Diagramme entité-relation de la base de données 1
1. Vue d'ensemble par domaines . . . 1
2. Diagramme ER détaillé . . . 2
3. Relations principales (verbalisées) . . . 5
4. Append-only et intégrité . . . 5
5. Indexes . . . 6
6. Tables hors périmètre cert . . . 6
7. Versionnement du schéma . . . 6

Diagramme entité-relation de la base de données

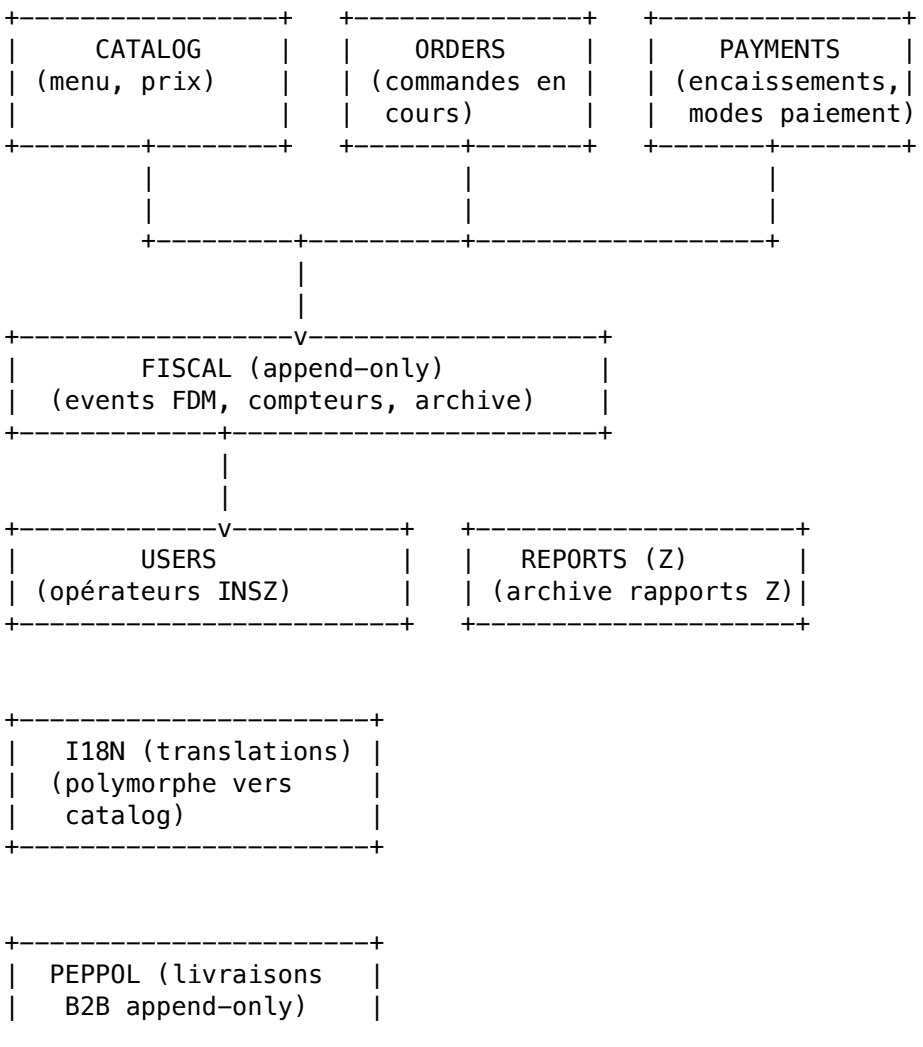
Date : 2026-05-14 Référence : §II.C "Relations entre les tables" du manuel de certification. Source de vérité : `crates/pos-db/src/schema.rs` (le SQL définit le schéma exact ; ce document reflète la révision actuelle).

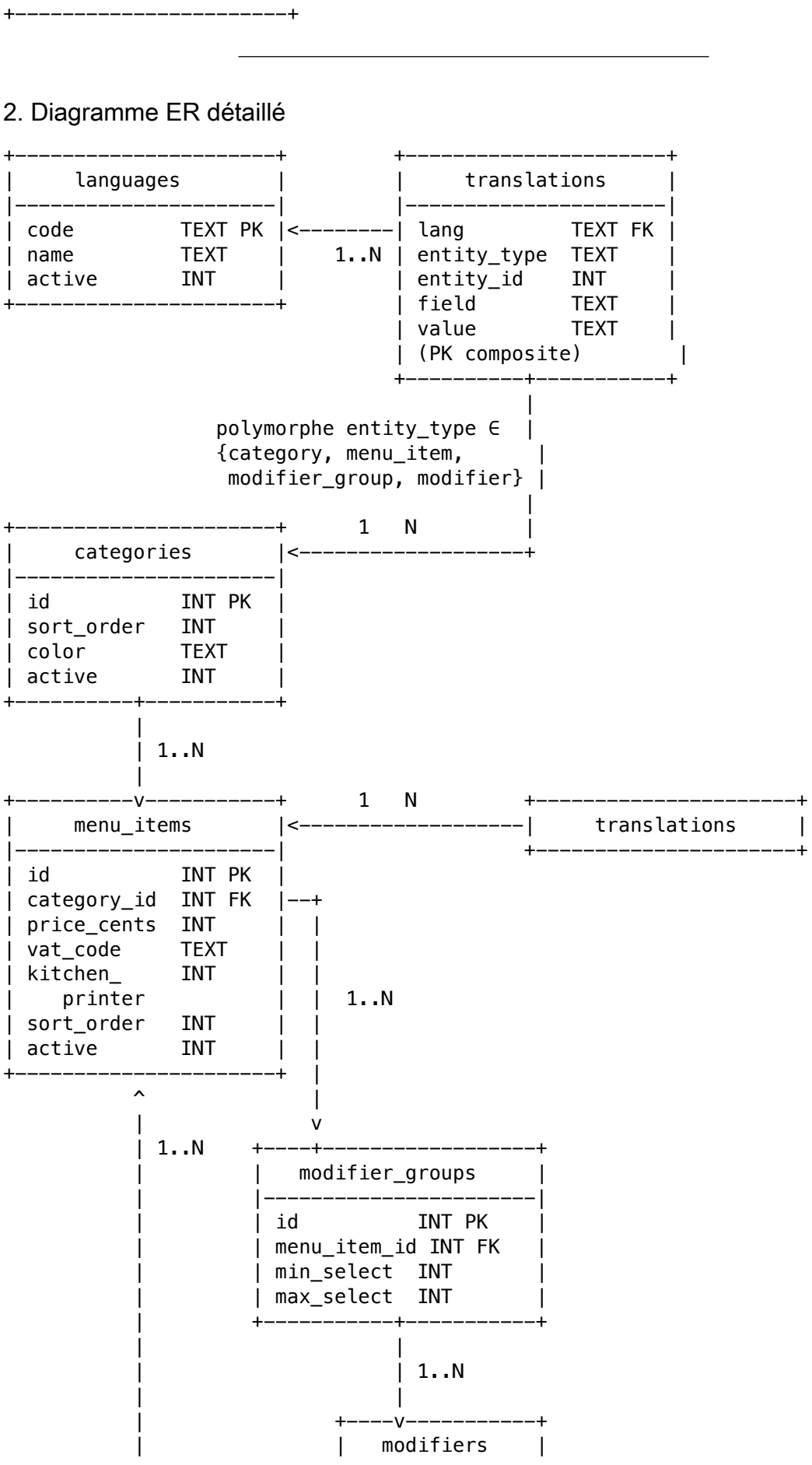
Ce document décrit visuellement les relations entre les tables de la base SQLite `pos.db`. Pour l'inventaire détaillé champ-par-champ, voir : SCHEMA_BD_FR.md. Pour la sécurité et les contraintes, voir : SECURITE_BD_FR.md.

---

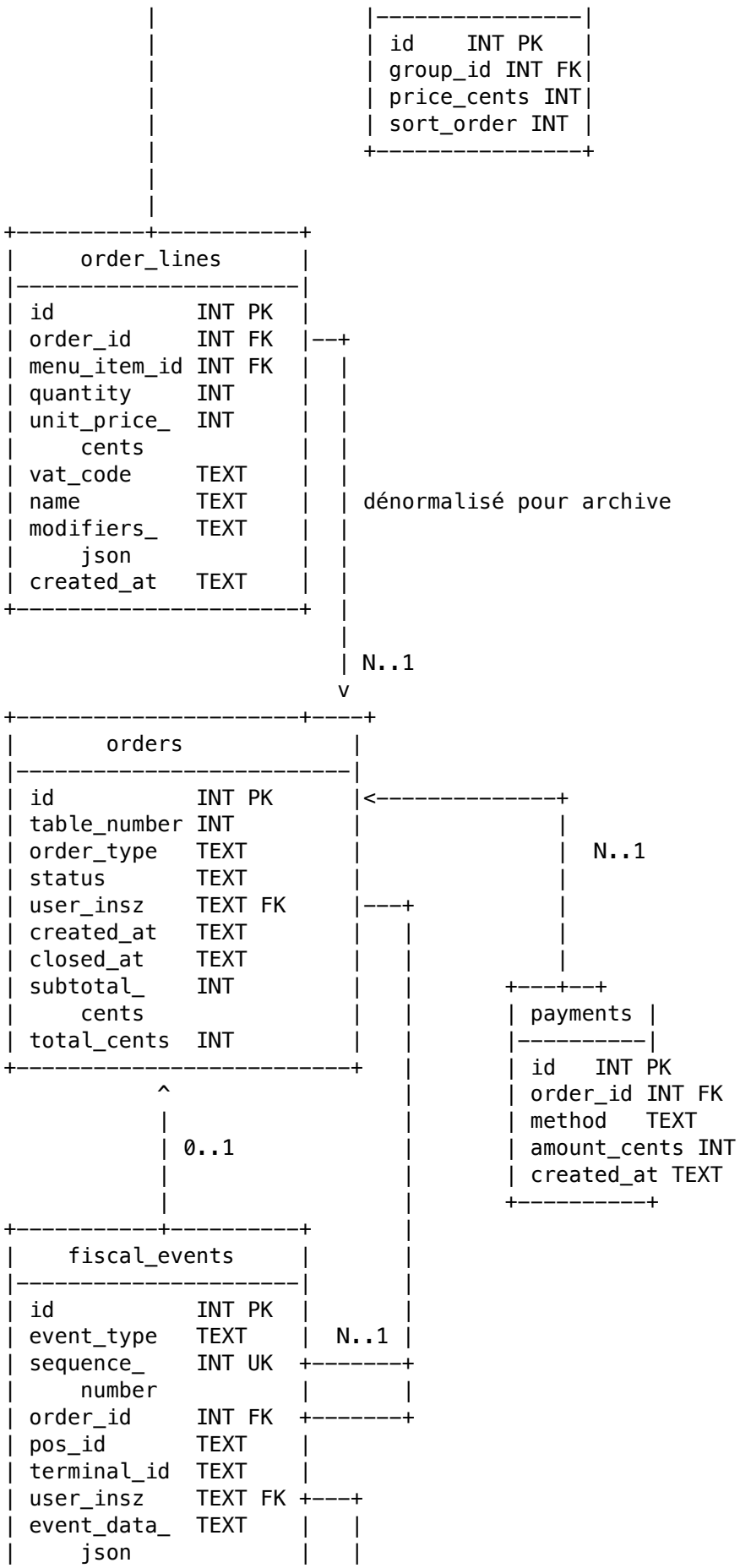
1. Vue d'ensemble par domaines

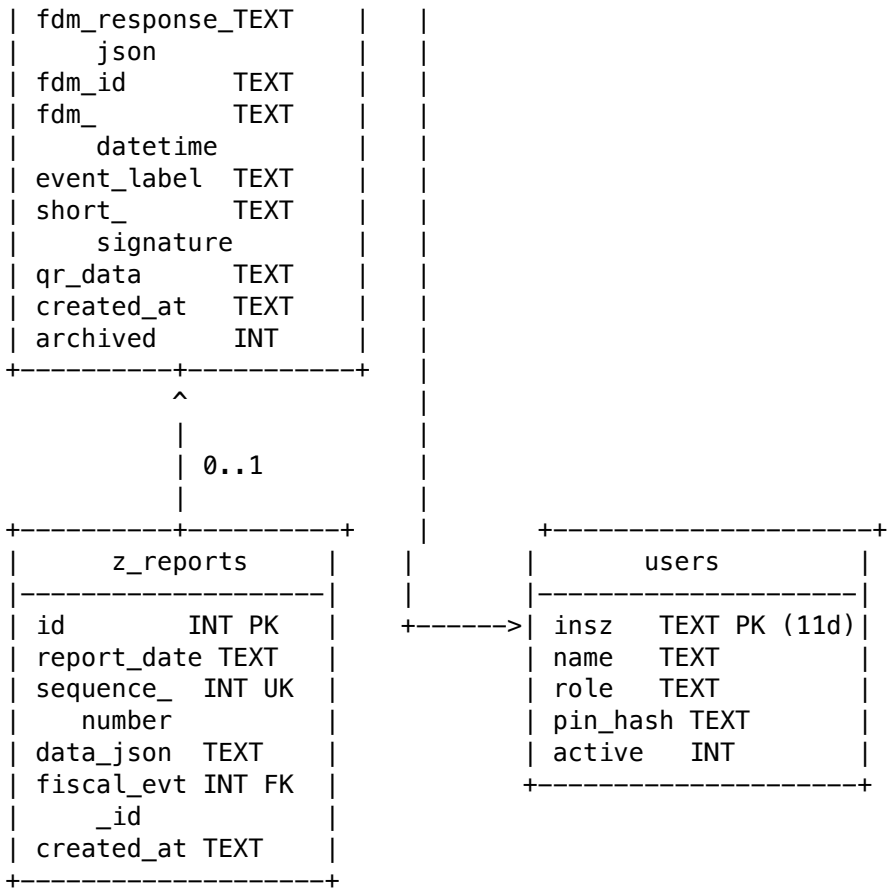
La base est organisée en 6 domaines logiques :

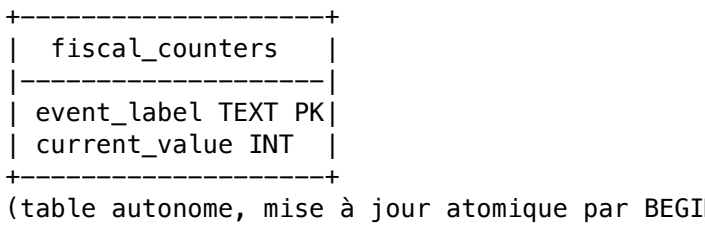




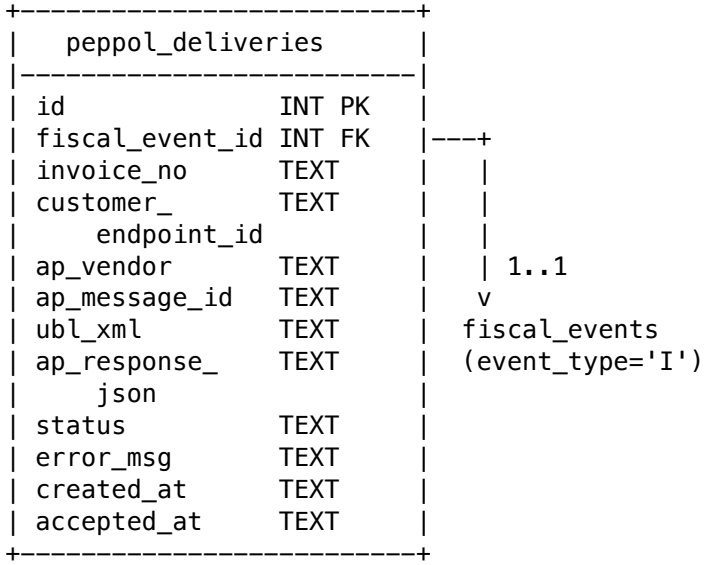
2. Diagramme ER détaillé


id INT PK
group_id INT FK
price_cents INT
sort_order INT
order_lines
id INT PK
order_id INT FK
menu_item_id INT FK
quantity INT
unit_price_cents INT
vat_code TEXT
name TEXT
modifiers_json TEXT
created_at TEXT
dénormalisé pour archive
N..1
orders
id INT PK
table_number INT
order_type TEXT
status TEXT
user_insz TEXT FK
created_at TEXT
closed_at TEXT
subtotal_cents INT
total_cents INT
N..1
payments
id INT PK
order_id INT FK
method TEXT
amount_cents INT
created_at TEXT
0..1
fiscal_events
id INT PK
event_type TEXT
sequence_number INT UK
order_id INT FK
pos_id TEXT
terminal_id TEXT
user_insz TEXT FK
event_data_json TEXT
N..1





(table autonome, mise à jour atomique par BEGIN IMMEDIATE
dans la même transaction qui ajoute à fiscal_events)



3. Relations principales (verbalisées)

De → Vers	Cardinalité	Sémantique
`categories` → `menu_items`	1 → N	Une catégorie contient N articles de menu
`menu_items` → `modifier_groups`	1 → N	Un article peut avoir N groupes de modificateurs (sauce, cuisson, ⋯)
`modifier_groups` → `modifiers`	1 → N	Chaque groupe contient N options (Soja, Wasabi, ⋯)
`orders` → `order_lines`	1 → N	Une commande contient N lignes
`order_lines` → `menu_items`	N → 1	Référence dénormalisée vers l'article (nom + prix copiés)
`orders` → `payments`	1 → N	Une commande peut être réglée par N paiements (split)
`orders` → `fiscal_events`	1 → 0..N	Une commande peut générer 0 (open) à plusieurs events fiscaux (P → N, ou N + C copie, etc.)
`fiscal_events` → `orders`	N → 0..1	Un event fiscal référence 0 (events système comme S, F-WAR) ou 1 commande
`users` → `orders`	1 → N	Un opérateur ouvre N commandes
`users` → `fiscal_events`	1 → N	Un opérateur émet N events (via `user_insz`)
`z_reports` → `fiscal_events`	1 → 1	Chaque Z archive un event R correspondant
`peppol_deliveries` → `fiscal_events`	1 → 1	Chaque livraison Peppol est liée à un event I
`translations` → (polymorphe)	N → 1	Traduction d'un champ d'une entité (category / menu_item / modifier_group / modifier) dans une langue (FR / NL / EN / ZH)
`translations` → `languages`	N → 1	Référence le code langue
`fiscal_counters` (autonome)	-	Pas de FK ; clé `event_label` ∈ {N, P, T, C, I, F, S, R}

4. Append-only et intégrité

Trois tables sont append-only au niveau applicatif (jamais de DELETE ni UPDATE) :

- `fiscal_events` - archive des events fiscaux signés par le FDM
- `z_reports` - archive des rapports de clôture
- `peppol_deliveries` - archive des livraisons B2B Peppol

Pour les autres tables (catalogue, commandes en cours, paiements, utilisateurs) les opérations UPDATE/DELETE sont autorisées via la couche commandes (avec audit trail dans les logs systemd-journald). Les `orders` sont marquées `status='closed'` ou `status='voided'` plutôt que supprimées.

La contrainte `fiscal_events.sequence_number UNIQUE` empêche tout doublon de compteur fiscal au niveau base. La table `fiscal_counters` est mise à jour atomiquement dans la même transaction

SQLite `BEGIN IMMEDIATE` que l'insert dans `fiscal_events` (cf. STRATEGIE_COMPTEURS_FR.md §5 - fix B3).

---

5. Indexes

Table	Index	Colonnes	Raison
`translations`	`idx_translations_lookup`	`entity_type, entity_id, lang`	Lookup rapide pour le rendu UI multilingue
`fiscal_events`	(implicite via PK + UNIQUE)	`id`, `sequence_number`	Lookup par ID et garantie d'unicité du compteur
`orders`	(implicite via PK)	`id`	Lookup par ID
`users`	(implicite via PK)	`insz`	Auth login

Des indexes additionnels peuvent être ajoutés en post-cert si des problèmes de performance apparaissent à plus grande échelle (multi-restaurant). Pour le périmètre TSB mono-terminal (~150 tickets/jour), aucun index supplémentaire n'est nécessaire.

---

6. Tables hors périmètre cert

Certaines tables existent dans le schéma mais sont inutilisées dans la build cert (intégrations tierces exclues par feature Cargo) :

— `hubrise_outbox` - queue d'événements webshop. Le schéma est préservé pour rétro-compatibilité de lecture, mais aucun chemin de code vivant ne l'écrit dans la build cert.

Ces tables apparaissent dans le SQL de seed mais ne reçoivent aucune écriture en production cert. Voir : RAPPORT_TESTS_BASELINE_FR.md § "Vérification de build" pour la preuve par `nm moyun-pos | grep -c pos_hubrise == 0`.

---

7. Versionnement du schéma

Le schéma est versionné via `pos-db::migrate::run_migrations`. La table `_migrations_applied` enregistre chaque migration appliquée (`version`, `name`, `applied_at`). Aucune modification de schéma post-cert n'est possible sans une nouvelle demande de certification (Article 109 du manuel).

Version actuelle du schéma : `v1` (initial cert build).